

Container Insights



Important

You must have completed the following chapters as pre-requisites for this lab:

- [Fundamentals](#)
- [ECS Service Connect](#)

By default, Amazon ECS provides metrics for clusters and services. However, by opting into Container Insights, you can enhance operational convenience by gaining access to additional metrics that facilitate operational management. [CloudWatch Container Insights](#) collects, aggregates, and summarizes metrics and logs from your containerized applications and microservices. You can also set CloudWatch alarms on metrics that Container Insights collects.

In this lab, we will first check whether Container Insights are enabled on your Amazon ECS Cluster. If Container Insights are not enabled, we will use AWS CLI to enable them for CloudWatch.

Check the status of Container Insights for the ECS cluster by filtering the AWS CLI output from [describe-clusters](#) . The output from the AWS CLI query will return the Container Insights status to indicate whether it is enabled or disabled for the ECS Cluster.

```
aws ecs describe-clusters --clusters retail-store-ecs-cluster --include SETTINGS --query 'clusters[*].settings'
```



► **Expand in case the Container Insight is not enabled.**

Previous

Next

Select your cookie preferences

We use essential cookies and similar tools that are necessary to provide our site and services. We use performance cookies to collect anonymous statistics, so we can understand how customers use our site and make improvements. Essential cookies cannot be deactivated, but you can choose “Customize” or “Decline” to decline performance cookies.

If you agree, AWS and approved third parties will also use cookies to provide useful site features, remember your preferences, and display relevant content, including relevant advertising. To accept or decline all non-essential cookies, choose “Accept” or “Decline.” To make more detailed choices, choose “Customize.”

Accept

Decline

Customize

Amazon ECS and AWS Fargate

- ▶ Getting Started
 - ▶ Fundamentals
 - ▶ Auto Scaling
 - ▶ Networking
 - ▼ Observability
 - ▼ Container Insights
 - Container Insights metrics
 - Explore CloudWatch logs
 - Load Testing ECS Microservices
 - ▶ OpenTelemetry
 - ▶ Security
 - ▶ Automation
 - ▶ Storage
- Congratulations!

Container Insights metrics

CloudWatch Container Insights collects metrics and logs from your containerized applications and microservices. It employs a containerized CloudWatch agent to discover and gather performance data from all running containers in a cluster. This data is then utilized to generate aggregated metrics at the cluster, service, and task levels, including resource utilization metrics for CPU, memory, disk, and network.

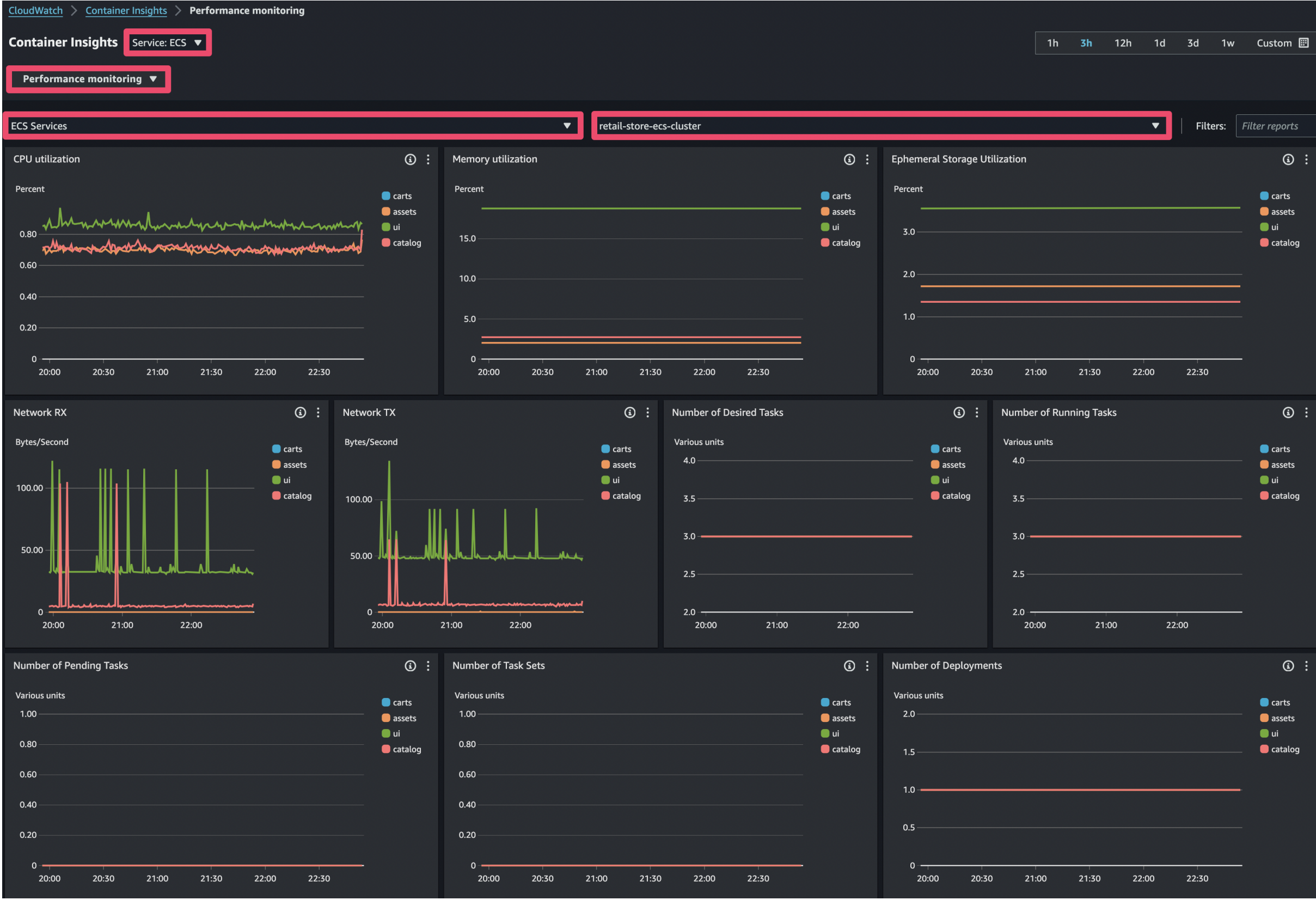
When you need regular statistics on aggregated data over time, it's preferable to use metrics rather than logs. Metrics are ideal for creating alarms or dashboard graphs.

Explore Container Insights metrics

To explore metrics, first open the CloudWatch Container Insights console:

Open Amazon CloudWatch console [↗](#)

In the CloudWatch console, select **Service: ECS** from the Service dropdown and **Performance Monitoring** from the second dropdown at the top. Then, choose the **retail-store-ecs-cluster**. Finally, change **ECS Cluster** to **ECS Services**.

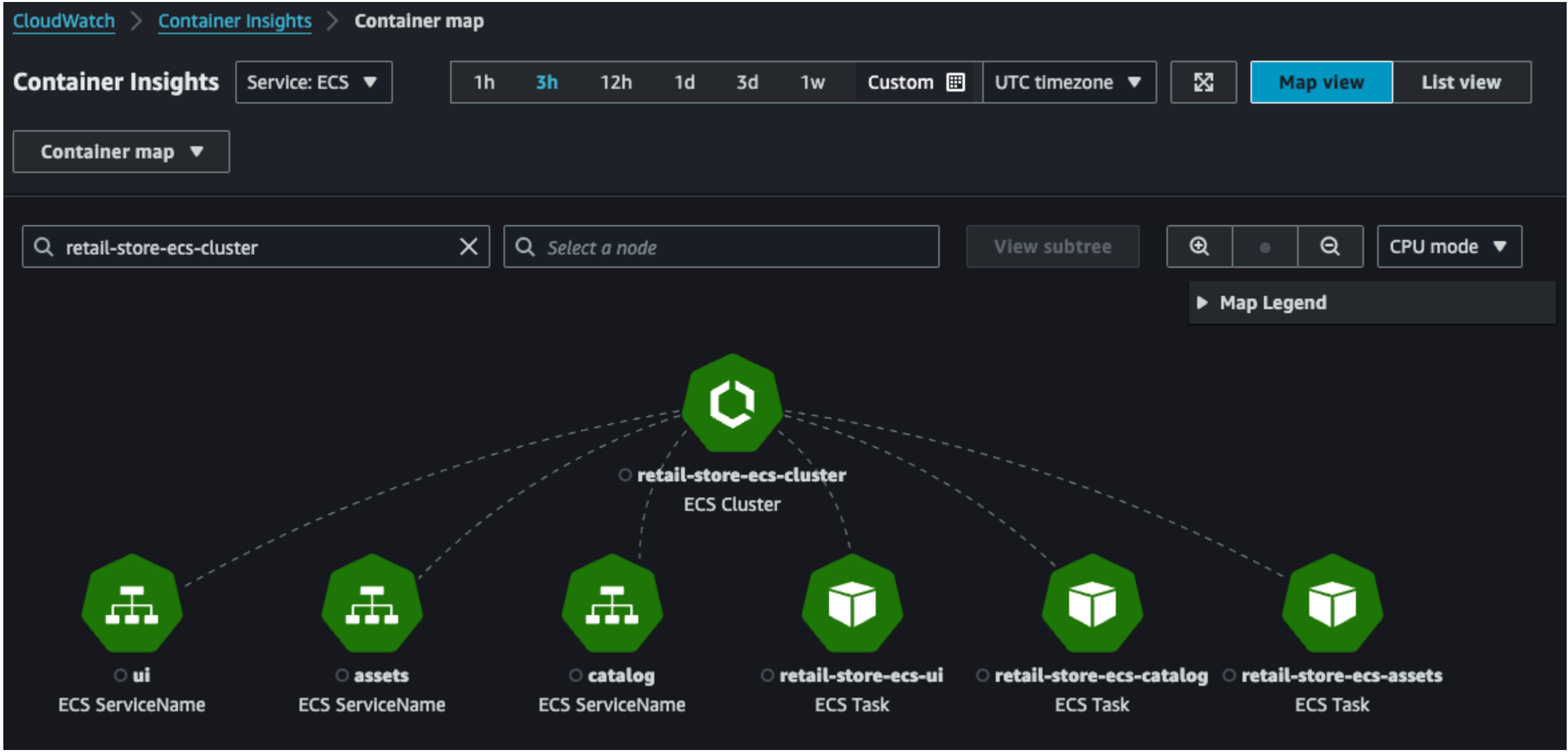


You can observe various metrics such as **CPU Utilization**, **Memory Utilization**, **Network RX/TX**, and **Ephemeral Storage Utilization**. These key metrics and automated dashboards are automatically generated by CloudWatch Container Insights based on the **retail-store-ecs-cluster**. Since these metrics are also available in CloudWatch Metrics, they can be used for creating custom dashboards as well.

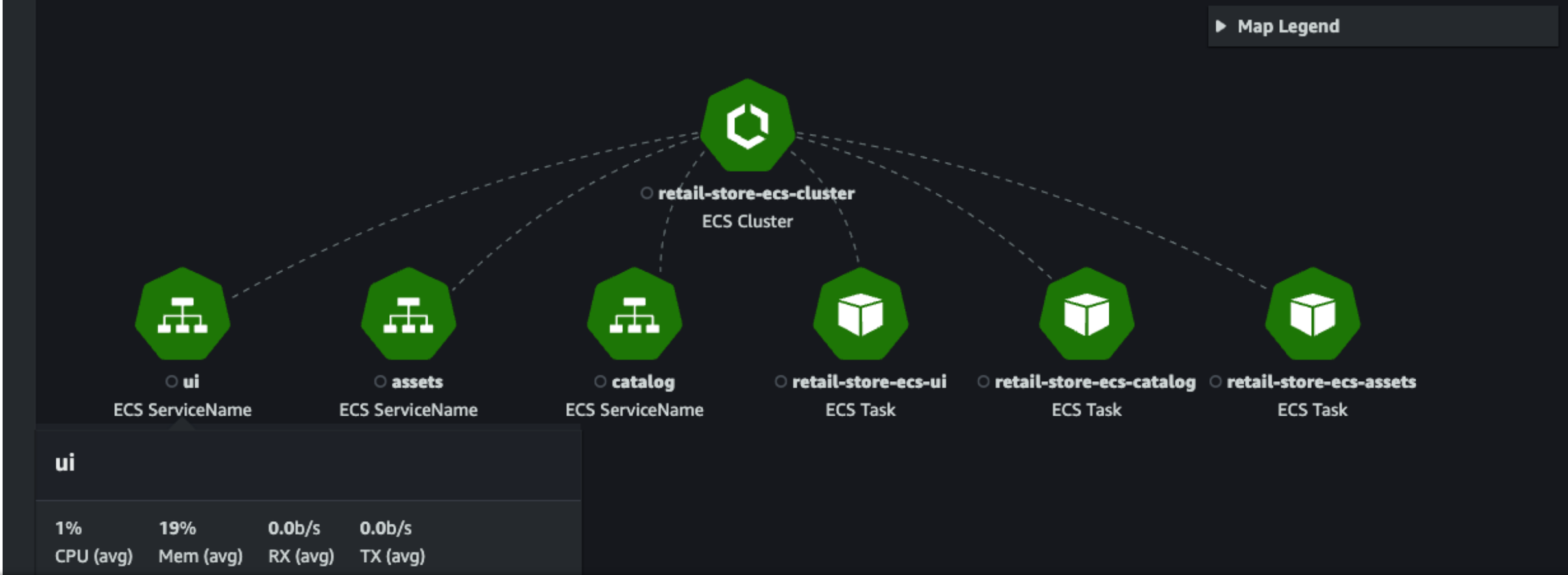
Amazon ECS Container Insights metrics are located in the ECS/ContainerInsights namespace. For detailed information about each metric, please refer to the [documentation here](#).

Explore application's visual map

To see a graphical representation of the ECS resources and their relationships, click on **View in maps** in the top right-hand side menu.



If you hover your mouse over one of the resources, you can view more details such as CPU, memory utilization, network traffic, and disk utilization.



Select your cookie preferences

We use essential cookies and similar tools that are necessary to provide our site and services. We use performance cookies to collect anonymous statistics, so we can understand how customers use our site and make improvements. Essential cookies cannot be deactivated, but you can choose “Customize” or “Decline” to decline performance cookies.

If you agree, AWS and approved third parties will also use cookies to provide useful site features, remember your preferences, and display relevant content, including relevant advertising. To accept or decline all non-essential cookies, choose “Accept” or “Decline.” To make more detailed choices, choose “Customize.”

Accept

Decline

Customize

aws

workshop studio

Amazon ECS Immersion Day

Getting Started

Fundamentals

Auto Scaling

Networking

Observability

Container Insights

Container Insights metrics

Explore CloudWatch logs

Load Testing ECS Microservices

OpenTelemetry

Security

Automation

Storage

Congratulations!

Amazon ECS Immersion Day > Observability > Container Insights > Load Testing ECS Microservices

Load Testing ECS Microservices

In this section, we'll generate synthetic load using the `stress` command, which is a command-line tool for Linux-based operating systems to introduce load into the system. We will `exec` into the `ui` task and run the `stress` command to trigger scaling. We will then observe the CloudWatch metrics to see the scaling effect.

Connect to the ECS Task

Run the following command to select one of the running tasks with `enableExecuteCommand` enabled:

```
ECS_EXEC_TASK_ARN=$(aws ecs list-tasks --cluster retail-store-ecs-cluster \
--service-name ui --query 'taskArns[]' --output text | \
xargs -n1 aws ecs describe-tasks --cluster retail-store-ecs-cluster --tasks | \
jq -r '.tasks[] | select(.enableExecuteCommand == true) | .taskArn' | \
head -n 1)

echo $ECS_EXEC_TASK_ARN
```

This will output the ARN of the task:

```
arn:aws:ecs:us-west-2:XXXXXXX:task/retail-store-ecs-cluster/0564778486a846599b8bd6b544e5f6eb
```

Start your `/bin/bash` interactive session in the running task.

```
if [ -z ${ECS_EXEC_TASK_ARN} ]; then echo "ECS_EXEC_TASK_ARN is not correctly configured!"; else
aws ecs execute-command --cluster retail-store-ecs-cluster \
--task $ECS_EXEC_TASK_ARN \
--container application \
--interactive \
--command "/bin/bash"
fi
```

Scaling Up

First, install the `stress` command:

```
dnf install stress -y
```

Now, issue the following command to stress 10 CPU cores with a timeout of 180 seconds:

```
stress -c 10 --timeout 180
```

Navigate to the Container Insights console page and select the time range to **5 minutes** as shown below:

Note

It will take a few minutes for the metrics to appear in CloudWatch Insights. To observe the stress spike in real-time, enable auto-refresh for 10 seconds in the CloudWatch console.

On the Container Insights console page, you can observe that the CPU utilization increases as the load on the `ui` service is increased.

After observing the scaling effect, terminate your session:

```
exit
```

Select your cookie preferences

We use essential cookies and similar tools that are necessary to provide our site and services. We use performance cookies to collect anonymous statistics, so we can understand how customers use our site and make improvements. Essential cookies cannot be deactivated, but you can choose “Customize” or “Decline” to decline performance cookies.

If you agree, AWS and approved third parties will also use cookies to provide useful site features, remember your preferences, and display relevant content, including relevant advertising. To accept or decline all non-essential cookies, choose “Accept” or “Decline.” To make more detailed choices, choose “Customize.”

Accept

Decline

Customize

